

A. Details on the Auto-CEn Algorithm

To improve the reproducibility of Auto-CEn, we show the pseudocode implementing our framework in Algorithm 6. The input of the algorithm consists of a dataset $\mathcal{D}$, the budgets b_{clf} and b_{df} for the classifier and decision fusion optimizations, and the maximum ensemble size k_{max} . The output is the optimal set of classifiers $\mathcal{C}^*$ and the optimal decision fusion method γ^* . The general procedure of Auto-CEn is given in lines 1-6 of Algorithm 6. Step 1 of Auto-CEn is implemented in lines 7-16, step 2 in lines 17-31, and step 3 in lines 32-44.

Note again that step 1 of Auto-CEn is not covered in Section IV-B, as it is based on a variant of state-of-the-art approaches to solve the CASH problem. So, we give a closer look at this first step in the following for reproducibility purposes.

1) *Step 1: Classifier Optimization:* Following the approach outlined in Section IV-A and shown in Figure 2, the first step is to overproduce a large number of accurate classifier configurations that can later be used as the basis for building the ensemble. Generally, it is unknown in advance which configurations in the configuration space Λ work well for a given dataset. Thus, optimizing the CASH problem allows for simultaneously evaluating numerous configurations and assessing their predictive performance. Unlike the result of the common CASH problem as defined in Equation 1, we are interested not only in the best classifier configuration but in a subset $\mathcal{C} \subseteq \Lambda$ of evaluated configurations whose predictive performance is better than random:

$$\mathcal{C} = \{\lambda^j | \lambda^j \in \Lambda, \mathcal{L}(\lambda^j, \mathcal{D}_{train}, \mathcal{D}_{valid}) < \mathcal{L}_{rand}\} \quad (1)$$

Here, $\mathcal{D}_{train}$ and $\mathcal{D}_{valid}$ are the train and validation split of the whole dataset $\mathcal{D}$, while $\mathcal{L}_{rand}$ is the value of the loss function $\mathcal{L}$ if all predictions were made at random. It is necessary to use only configurations that are better than random, as they may otherwise reduce the ensemble's predictive performance instead of improving it [1]. To evaluate the configurations, we use cross-validation, as this is typically used by existing approaches as well in order to reduce the risk of getting overfitted classifier configurations [2, 3].

Note that the number of classifier configurations returned in the set $\mathcal{C}$ is usually higher than the size k of the ensemble. So, this first step overproduces a large set of classifier configurations, which is then reduced to k configurations in the subsequent step.

B. Experimental Setup

Details about the experimental setup described in Section V-A are given below. This includes information about the software and hardware, the datasets, configuration spaces and the configuration of the baselines.

Fig. 6: Auto-CEn

Input: $\mathcal{D}$: Dataset, b_{clf} : Budget Classifier Optimization, b_{df} : Budget Decision Fusion Optimization, k_{max} : Maximum Ensemble Size
Output: $\mathcal{C}^*$: Classifier Set, γ^* : Decision Fusion

```

1: procedure TRAINENSEMBLE( $\mathcal{D}, b_{clf}, b_{df}, k_{max}$ )
2:    $\mathcal{C} \leftarrow \text{OPTIMIZECLASSIFIERS}(\mathcal{D}, b_{clf})$ 
3:    $\mathcal{C}_{all}^* \leftarrow \text{CREATECLASSIFIERSET}(\mathcal{C}, \mathcal{D}, k_{max})$ 
4:    $\mathcal{C}^*, \gamma^* \leftarrow \text{OPTIMIZEFUSION}(\mathcal{C}_{all}^*, \mathcal{D}, b_{df}, k_{max})$ 
5:   return  $\mathcal{C}^*, \gamma^*$ 
6: end procedure

7: procedure OPTIMIZECLASSIFIERS( $\mathcal{D}, b_{clf}$ )
8:    $\Lambda \leftarrow \text{init\_clf\_configuration\_space}()$ 
9:    $\mathcal{C} \leftarrow \text{optimize\_CASH}(\mathcal{L}, \Lambda, \mathcal{D}, b_{clf})$ 
10:  for all  $\lambda_j \in \mathcal{C}$  do
11:    if  $\mathcal{L}(\lambda_j, \mathcal{D}) < \mathcal{L}_{rand}$  then
12:       $\mathcal{C}.\text{remove\_entry}(\lambda_j)$ 
13:    end if
14:  end for
15:  return  $\mathcal{C}$ 
16: end procedure

17: procedure CREATECLASSIFIERSET( $\mathcal{C}, \mathcal{D}, k_{max}$ )
18:    $\mathcal{C}_{all}^* \leftarrow \emptyset$ 
19:    $\text{DM} \leftarrow \text{compute\_diversity}(\mathcal{C}, \mathcal{D})$ 
20:    $\text{tree} \leftarrow \text{agglomerative\_clustering}(\text{DM})$ 
21:   for  $k = 2, \dots, k_{max}$  do
22:      $\mathcal{C}_k^* \leftarrow \emptyset, \text{clu}_k \leftarrow \text{tree}(k)$ 
23:      $\triangleright$  Iterate over the  $k$  clusters
24:     for  $i = 1, \dots, k$  do
25:        $c_i \leftarrow \text{max\_perf}(\text{clu}_k[i])$ 
26:        $\mathcal{C}_k^* \leftarrow \mathcal{C}_k^* \cup \{c_i\}$ 
27:     end for
28:      $\mathcal{C}_{all}^* \cup \{\mathcal{C}_k^*\}$ 
29:   end for
30:   return  $\mathcal{C}_{all}^*$ 
31: end procedure

32: procedure OPTIMIZEFUSION( $\mathcal{C}_{all}^*, \mathcal{D}, b_{df}, k_{max}$ )
33:    $\gamma^* \leftarrow \text{Empty}, \mathcal{C}^* \leftarrow \emptyset$ 
34:    $\Gamma \leftarrow \text{init\_df\_configuration\_space}()$ 
35:   for  $k = 2, \dots, k_{max}$  do
36:      $\mathcal{C}_k^* \leftarrow \mathcal{C}_{all}^*[k]$ 
37:      $\mathcal{Y}_{df} = \text{create\_fusion\_data}(\mathcal{C}_k^*, \mathcal{D})$ 
38:      $\gamma_k^* \leftarrow \text{optimize\_CASH}(\mathcal{L}, \Lambda, \mathcal{Y}_{df}, b_{df})$ 
39:     if  $\mathcal{L}(\gamma_k^*, \mathcal{Y}_{df}) < \mathcal{L}(\gamma^*, \mathcal{Y}_{df})$  then
40:        $\gamma^* \leftarrow \gamma_k^*, \mathcal{C}^* \leftarrow \mathcal{C}_k^*$ 
41:     end if
42:   end for
43:   return  $\mathcal{C}^*, \gamma^*$ 
44: end procedure

```

TABLE III: All 20 datasets used in the evaluation, sorted by their number of instances. The ID denotes the OpenML ID of the datasets. NUM = Numerical features, CAT = Categorical features, MIX = Numerical and Categorical features.

ID	41156	40983	40701	1489	4538	1459	40926	6	184	4135	41027	1461	1590	41166	40927	40668	40996	554	41168	42769
	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13	D14	D15	D16	D17	D18	D19	D20
# Instances	4147	4839	5000	5404	9873	10218	20000	20000	28056	32769	44819	45211	48842	58310	60000	67557	70000	70000	83733	1000000
# Features	48	5	20	5	32	7	3072	16	6	9	6	16	14	180	3072	42	784	784	54	28
# Classes	2	2	2	2	5	10	10	26	18	2	3	2	2	10	10	3	10	10	4	2
Feature Types	NUM	NUM	MIX	NUM	NUM	NUM	NUM	NUM	CAT	CAT	NUM	MIX	MIX	NUM	NUM	CAT	NUM	NUM	NUM	NUM

TABLE IV: Diversity results of the diversity ablation described in Section V-C1. The diversity is measured by the double fault (DF), correlation coefficient (Corr.) and disagreement metrics (Dis.) for both Auto-CEn and the ablated system. All values are adjusted so that a high value denotes high diversity.

		D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
Auto-CEn	1-DF	0.90±0.01	0.99±0.00	0.96±0.00	0.94±0.01	0.80±0.03	0.95±0.00	0.49±0.03	0.98±0.00	0.91±0.03	0.95±0.01
	1-Corr.	0.40±0.11	0.36±0.15	0.22±0.08	0.48±0.07	0.53±0.15	0.17±0.06	0.61±0.16	0.39±0.15	0.62±0.19	0.22±0.15
	Dis.	0.11±0.04	0.01±0.01	0.02±0.01	0.09±0.01	0.23±0.06	0.02±0.01	0.28±0.07	0.02±0.01	0.19±0.07	0.03±0.02
Ablated System	1-DF	0.80±0.02	0.99±0.00	0.96±0.00	0.92±0.01	0.77±0.03	0.95±0.00	0.45±0.03	0.98±0.00	0.90±0.02	0.94±0.01
	1-Corr.	0.06±0.07	0.33±0.08	0.08±0.02	0.12±0.04	0.38±0.16	0.13±0.02	0.32±0.16	0.42±0.11	0.50±0.06	0.16±0.11
	Dis.	0.02±0.02	0.01±0.00	0.01±0.00	0.02±0.01	0.16±0.07	0.01±0.00	0.15±0.08	0.02±0.01	0.15±0.02	0.02±0.02

		D11	D12	D13	D14	D15	D16	D17	D18	D19	D20
Auto-CEn	1-DF	0.93±0.02	0.94±0.01	0.91±0.01	0.75±0.02	0.41±0.11	0.89±0.01	0.89±0.01	0.96±0.01	0.77±0.01	0.81±0.03
	1-Corr.	0.44±0.15	0.47±0.21	0.47±0.20	0.39±0.09	0.56±0.41	0.33±0.08	0.36±0.08	0.49±0.12	0.46±0.17	0.64±0.22
	Dis.	0.09±0.03	0.11±0.09	0.14±0.07	0.18±0.04	0.25±0.18	0.08±0.02	0.10±0.02	0.07±0.03	0.21±0.09	0.30±0.12
Ablated System	1-DF	0.93±0.00	0.86±0.05	0.89±0.01	0.75±0.01	0.41±0.03	0.89±0.01	0.88±0.01	0.96±0.01	0.75±0.01	0.79±0.01
	1-Corr.	0.39±0.04	0.40±0.15	0.21±0.11	0.35±0.06	0.36±0.31	0.25±0.06	0.27±0.09	0.36±0.11	0.38±0.15	0.49±0.01
	Dis.	0.08±0.01	0.13±0.04	0.06±0.04	0.16±0.03	0.16±0.14	0.06±0.02	0.07±0.02	0.04±0.02	0.18±0.08	0.21±0.01

1) *Soft- and Hardware:* We implemented our own AutoML framework for Auto-CEn in Python version 3.10 using scikit-learn [4] for the classification algorithms and PUSION [5] for the decision fusion algorithms. The CASH optimizations used in steps 1 and 3 for the classifier optimization and for the decision fusion optimization are implemented using the SMAC3 [6] library. The agglomerative clustering algorithm of our post hoc method in step 2 is implemented via the SciPy library [7]. We conducted the experiments on a virtual machine running Ubuntu 22.04 with 8 VCPU cores and 16GB RAM. For the baselines we used the implementations provided by the respective authors.

2) *Datasets:* The evaluation was carried out using the 20 datasets obtained from the OpenML repository [8]. The datasets were selected according to their wide range of data characteristics, such as the number of data instances, features and classes, as well as the feature types. All of these characteristics are given in Table III.

3) *Configuration Space:* The classifier configuration space of our framework Auto-CEn consists of the same 15 classification algorithms and hyperparameters as used by auto-sklearn [3], including, among others, state-of-the-art ensemble algorithms such as Random Forest and Gradient Boosting. The decision fusion configuration space consists of 13 decision fusion algorithms provided by PUSION [5].

4) *Baselines:* The baselines were executed with the same settings as Auto-CEn if possible (cf. Section V-A2). In the following, the settings used for each baseline are detailed.

auto-sklearn: For auto-sklearn, we use the same settings as for Auto-CEn, that is, one hour as the optimization budget, the balanced error rate as the loss metric, and 10-fold cross-

validation for the evaluation of classifier configurations. We also use the same dataset split of 80% training data and 20% test data as described in Section V-A1. To allow for a fair comparison, we disable the data pre-processing features in the auto-sklearn baseline. The rationale behind this is that we are exclusively interested in evaluating and comparing the abilities to optimize classifier ensembles and the decision fusion. So, we need to exclude side effects resulting from data pre-processing that make comparing and interpreting the results less comprehensible.

QDO-ES: As the implementation provided for QDO-ES uses auto-sklearn to create its base classifiers, we apply the same settings to QDO-ES as for auto-sklearn. Furthermore, we use QDO-ES with their SiloTopN approach, as it was reported to work best [9].

DivBO: DivBO is also used with the same settings as described above, with the exception that we do not use 10-fold cross-validation for the evaluation of classifier configurations, as DivBO does not natively support this and instead uses the holdout method.

C. Detailed Results of the Ablation Study

More details on the results of the ablation study are given in this section. To allow for a closer look at the results of the ablation study omitting the diversity optimization (see Section V-C1), Table IV summarizes the results of three diversity metrics for Auto-CEn and for the ablated system. Here, we see that the trends regarding different diversity metrics are very similar to the double fault metric used in Section V-C1. This reinforces the major outcomes of our discussion.

REFERENCES

- [1] L. K. Hansen and P. Salamon, “Neural network ensembles,” *IEEE Trans. on Pattern Analysis and Machine Intelligence*, vol. 12, no. 10, pp. 993–1001, 1990.
- [2] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, “Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms,” in *Proc. of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Chicago Illinois USA: ACM, aug 2013, pp. 847–855.
- [3] M. Feurer *et al.*, “Efficient and robust automated machine learning,” in *Advances in Neural Information Processing Systems*, C. Cortes *et al.*, Eds., vol. 28. Curran Associates, Inc., 2015.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, nov 2011.
- [5] Y. Wilhelm, P. Reimann, W. Gauchel, S. Klein, and B. Mitschang, “Pusion - A generic and automated framework for decision fusion,” in *Proc. of the 39th International Conference on Data Engineering (ICDE)*. Anaheim, CA, USA: IEEE, apr 2023, pp. 3282–3295.
- [6] M. Lindauer *et al.*, “Smac3: A versatile bayesian optimization package for hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.
- [7] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in python,” *Nature Methods*, vol. 17, pp. 261–272, mar 2020.
- [8] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, “Openml: Networked science in machine learning,” *SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, jun 2014.
- [9] L. Purucker *et al.*, “Q(D)O-ES: population-based quality (diversity) optimisation for post hoc ensemble selection in automl,” in *Proc. of the 2nd International Conference on Automated Machine Learning*, A. Faust *et al.*, Eds., vol. 224. PMLR, nov 2023, pp. 1/1–23.